

Manual for LPC3D based Pystencils (LPC3D.py) and multiscale coupling between LPC3D and LAMMPS (LPC3D-LAMMPS.py)

Contents

1	Introduction	2
2	Installation	2
2.1	Desktop and HPC cluster	2
2.2	EESSI project	3
3	Running the program	3
4	Formats of input file	3
4.1	“Bulk” type simulations	3
4.2	“electrode” type simulations	4
4.3	“supercapacitor” type simulations	4
5	Multiscale coupling between LPC3D and LAMMPS	6
5.1	Principle	6
5.2	Installation	6
5.3	Required files in the working directory	7
5.4	Running the program	7
5.5	Format of the input file	8
5.6	Output files	9

1 Introduction

Over the past decade, most of the simulations of supercapacitors were performed at the microscopic scale, using molecular dynamics software such as LAMMPS and ESPResSo. This allowed to understand the adsorption of ions and the effect of surface porosity on some electrochemical properties. However it is well known from experiments that commercial materials are highly inhomogeneous. For example, the typical pore size distributions range from 0.8 to 2 nm. Molecular simulations, where electrode sizes are a few nanometers, only allow for the inclusion of a few pores. It is therefore necessary to simulate electrodes and supercapacitors at larger scales. To this end, LPC3D is a software designed for mesoscopic simulations of capacitive properties of carbon-carbon capacitors, based on a lattice gas model. The code calculates quantities of adsorbed ions, diffusion coefficients and predicts NMR spectra of ions adsorbed in porous carbon matrices. It uses information from molecular dynamics simulations as its main input, so that the parameterization is made at the microscale.

The initial LPC3D code, which was written in C, was a serial code. It allowed routine simulations of systems on a $30 \times 30 \times 30$ grid, that corresponds to a dimension of approximately $(1 \mu\text{m})^3$, i.e. the size of a single - relatively small - particle, much smaller than an electrode and than a supercapacitor device.

The migration from the serial code to Python code based PyStencils has been performed. PyStencils was chosen since it allows to generate highly efficient and scalable C++ and CUDA/HIP code from the mathematical description of the model. This now allows one to run the model in parallel using OpenMP on CPU or hybride CPU-GPU, making use of a full node. This new implementation allows to simulate systems on a $100 \times 100 \times 100$ grid (1 million sites), with a very short simulation time of around 0.5 seconds per timestep. It is therefore now possible to simulate routinely systems with several small particles immersed in a bulk electrolyte, electrodes and as well as a full supercapacitor.

Key Features

- Symbolic expressions for defining simulation equations
- Support for high-performance stencil computations
- CPU parallelization using OpenMP
- Hybride CPU-GPU

2 Installation

2.1 Desktop and HPC cluster

In this model, carbon electrode, fluid and supercapacitor are simulated. The lattice structure is represented as inter-connected discrete sites, separated by a lattice spacing.

Before running the code, several modules need to be installed on desktop or HPC cluster; if you want to run the software on CPU, you have to install pystencils, numba, matplotlib, Ipython and vtk, using pip with the command:

```
pip install pystencils numba Ipython matplotlib pyevtk
```

If you want to run the software on GPU, you need to install additional modules, such as cupy (make sure to match the module version with your CUDA version) using pip with the command:

```
pip install cupy-cudaXXX
```

And also load CUDA module with the following command line:

```
module load cuda/XXX
```

Alternatively, LPC3D can also be installed directly as a Python package from PyPI using:

```
pip install lpc3d
```

This command installs LPC3D and its Python dependencies into your current Python environment. Once installed, you can use LPC3D according to your local setup.

2.2 EESSI project

LPC3D is available in EESSI project. Check the '<https://www.eessi.io/>' for more information about siting up and installation of EESSI.

3 Running the program

On CPU or hybride CPU-GPU, the program can be run using the following command line:

```
OMP_NUM_THREADS=NUMBER_OF_THREADS python3 LPC3D.py -i filename
```

4 Formats of input file

- 1st line: type of processing unit to be used for the computation ($> \text{cpu}$ or gpu)
- 2st line: numbers of lattice sites in x , y and z directions ($> N_x, N_y, N_z$)
- 3st line: boudary conditions in x , y and z directions, true or false ($> \text{true true true}$)
- 4nd line: lattice parameter in Å ($> a$)
- 5nd line: dwell time in seconds ($> \text{dwell}$)
- 6th line: number of steps between measures ($> \text{nsample}$)
- 7th line: temperature in Kelvin ($> T$)
- 8th line: Larmor frequency of the studied nucleus under the magnetic field considered in MHz ($> \text{larmorfreq}$)
- 9th line: number of values for the Fourier transform ($> N_{\text{values}}$)
Please use an even number of values.
- 10th line: energy barrier value ($> E_{\text{bar}}$)
- 11th line: equilibration process. 0 if no, 1 if yes ($> \text{equi}$)
- 12th line:
 - no 12th line if $\text{equi}=0$
 - if $\text{equi}=1$: number of steps for the equilibration process ($> N_{\text{step}}$)
- 13th line: electrode nature (porous-carbon or particles) ($> \text{elec} - \text{nature}$)
 - if elec-nature is particles, give a xyz file for "B" (bulk) and "P" particles ($> \text{partcilefile}$)
- 14th line: number of blocks ($> N_{\text{blocks}}$)
 - We divide the simulation lattice into blocks along the z-axis.
 - PS: All blocks should have the same size.

After this line, the uploaded files differ depending on the block nature. Block nature: bulk or electrode.

4.1 "Bulk" type simulations

if $N_{\text{blocks}} = 1$:

- 15th line: An empty line
- 16th line: block nature ($> \text{bulk}$)
- 17th line: $x_{\text{min}}, x_{\text{max}}, y_{\text{min}}, y_{\text{max}}, z_{\text{min}}, z_{\text{max}}$ coordinates of the block
- 18th line: density of the bulk ($> \text{dens}_{\text{bulk}}$)
- 19th line: frequency of the bulk (in ppm) ($> N_{\text{freq}}$)

```

cpu
50 50 50
true true true
1.0
5e-6
1
298
282
300
0.0
1
50000
1

bulk
1 50
1 50
5 50
0.296297
0.0000

```

Input example for “bulk” type simulations.

4.2 “electrode” type simulations

if $Nblocks = 1$:

- 15th line: An empty line
- 16th line: block nature ($> electrode$)
- 17th line: electrode is filled with fluid or not? 0 if no, 1 if yes ($> fielectrode$)
- 18th line: xmin,xmax,ymin,ymax,zmin,zmax coordinates of the block
- 19th line: name of the file with pore size distribution ($> name_{psd}$) The first column should be in Å, the second column is the probability.
- 20th line: name of the file with pores size as a function of density and frequency ($> name_{poresize_{dens_{freq}}}$) The first column should be in Å, the second column is the densities of the fluid and the third column is the frequencies and should be in ppm.

```

cpu
50 50 50
true true true
1.0
5e-6
1
298
282
300
0.0
1
50000
1

electrode
1
1 50
1 50
1 50
psd.txt
pore_dens_freq_2neg.txt

```

Input example for “electrode” type simulations.

4.3 “supercapacitor” type simulations

simulation of supercapacitor is actually a combination of bulk and electrode constituting 3 blocs, (electrode/bulk/electrode). Here is an input example that illustrates the simulation of supercapacitor. PS: don’t forget to make an empty line between blocks.

```

cpu
50 50 150
true true false
1,0
5e-6
1
298
282
300
0.0
1
50000
3

electrode
1
1 50
1 50
1 50
psd.txt
pore_dens_freq_2neg.txt

bulk
1 50
1 50
51 100
0.296297
0.000

electrode
1
1 50
1 50
101 150
psd.txt
pore_dens_freq_2pos.txt

```

Input example for “supercapacitor” type simulations.

5 Multiscale coupling between LPC3D and LAMMPS

5.1 Principle

In the coupled version, the pore-size-dependent densities of the confined fluid are no longer read from a file but computed by molecular dynamics with LAMMPS, called through its Python interface. Both codes run within a single MPI job and exchange data with `mpi4py`. The workflow is the following:

1. The rank 0 process reads the input file and extracts the list of distinct pore sizes from the pore size distribution file. This list, together with the names of the molecular files and LAMMPS input scripts, is broadcast to all MPI processes.
2. The global MPI communicator is split into one sub-communicator per pore size. Each sub-communicator runs an independent LAMMPS simulation of the fluid confined in a slit pore of the corresponding width.
3. For each pore size, a dedicated folder `simulation_<pore_size>` is created. The two carbon walls are displaced to obtain the desired pore width, the simulation box is built with `fftool` and `Packmol`, and LAMMPS performs an equilibration run followed by a production (NVT) run in the NVT subfolder.
4. The centres of mass of the molecules are computed from the NVT trajectory, and the number densities of each species are evaluated inside the pore, in two control regions corresponding to the two electrodes.
5. The densities of all pore sizes are gathered on rank 0, broadcast back to all processes, and written to `all_densities.dat`.
6. LPC3D assigns to each lattice site of the electrode blocks a pore size drawn from the pore size distribution, together with the density computed by LAMMPS and the chemical shift read from the pore-frequency file for this pore size. The lattice gas simulation then proceeds as in the standalone version.

When GPUs are used, the LAMMPS simulations are run in successive waves, with one simulation per available GPU over all the nodes of the job.

5.2 Installation

In addition to the dependencies of `LPC3D.py`, the coupled version requires:

- **mpi4py**, compiled against the same MPI library as LAMMPS:

```
pip install mpi4py
```

- **LAMMPS built as a shared library with its Python module**, for example with CMake:

```
cmake -D BUILD_SHARED_LIBS=on -D BUILD_MPI=on -D PKG_PYTHON=on [other packages] ../cmake
make -j N && make install-python
```

The packages required by your LAMMPS input scripts (force field, constraints, etc.) must also be enabled. The installation can be checked with `python3 -c "from lammps import lammps; lammps()"`.

- **For GPU runs**, LAMMPS must be compiled with the KOKKOS package and CUDA support (`-D PKG_KOKKOS=on -D Kokkos_ENABLE_CUDA=on` and the flag corresponding to your GPU architecture).
- **fftool** and **Packmol**, which are used to build the initial configurations. Their executables must be named `fftool` and `packmol` and placed in the working directory.

5.3 Required files in the working directory

File	Description
LAMMPS-LPC3D.py	Main program (LPC3D driver).
mpi4py_lammps.py	Python module handling the LAMMPS simulations; it must be importable by the main program.
lattice_gas.inpt	Input file (Section 5.5).
CG1.xyz, CG2.xyz	Coordinates of the two carbon walls. They are shifted along x by $\pm d/2$, where d is the pore size.
*.zmat	Molecular description of the three fluid species (anion, cation and electrolyte), used by fftool.
il.ff	Force field file used by fftool.
fftool, packmol	Executables used to build the simulation box.
sites2cm.inpt	Parameters for the centre-of-mass calculation (see below).
LAMMPS inputs	Equilibration and production (NVT) scripts, named in the input file.
PSD file	Pore size distribution: pore size in Å (column 1), probability (column 2).
Pore-frequency files	One per electrode block: pore size in Å (column 1), then the chemical shift in ppm of species 1, 2 and 3 (columns 2 to 4).

The following constraints must be respected:

- The pore sizes of the pore-frequency files must be the same as the distinct pore sizes of the PSD file, sorted in increasing order, with one line per pore size.
- The equilibration script must write the final configuration with `write_data data.*.lmp`, since the program looks for a file named `data.<step>.lmp` to start the NVT run.
- The NVT script must write the trajectory of the fluid in xyz format to `positions.xyz`. Only the fluid atoms must be dumped (no carbon atoms), grouped by species in the order of the zmat files, with the atoms of each molecule written consecutively (e.g. with `dump_modify sort id`).

The file `sites2cm.inpt` has the following format, one value per line:

1. file type (2 for an xyz trajectory with atom labels);
2. number of configurations in `positions.xyz`;
3. number of molecular species (3);
4. maximum number of atoms per molecule;
5. then, for each species: number of atoms per molecule, number of molecules, and the mass of each atom of the molecule (one per line).

5.4 Running the program

The number of MPI processes must be greater than or equal to the number of distinct pore sizes in the PSD file, since each pore size is simulated by at least one MPI process. The processes are distributed as evenly as possible among the simulations.

With SLURM:

```
export OMP_NUM_THREADS=<N_THREADS> #for LPC3D part
export LPC3D_NUM_GPUS=<N_GPUS>      # only if GPUs are used
srun python3 LAMMPS-LPC3D.py -i lattice_gas.inpt
```

Or with mpirun:

```
export OMP_NUM_THREADS=<N_THREADS> #for LPC3D part
export LPC3D_NUM_GPUS=<N_GPUS>      # only if GPUs are used
mpirun -np <N_MPI_PROCS> python3 LAMMPS-LPC3D.py -i lattice_gas.inpt
```

LPC3D_NUM_GPUS is the number of GPUs per node (1 by default). OMP_NUM_THREADS sets the number of threads used by the lattice part of the calculation.

5.5 Format of the input file

The first lines of the input file are identical to those of the standalone version (lines 1 to 13 of Section 4). The differences concern the pore size distribution, the description of the blocks and the additional parameters for LAMMPS. The complete structure is the following:

- Lines 1 to 13: as in the standalone version (processing unit, lattice size, boundary conditions, lattice parameter, dwell time, **nsample**, temperature, Larmor frequency, **Nvalues**, energy barrier, equilibration, number of equilibration steps if **equi**=1, electrode nature, and particle file if the electrode nature is **particles**).
- Next line: name of the pore size distribution file. In the coupled version this file is given only once, for all electrode blocks, since it defines the pore sizes simulated with LAMMPS.
- Next line: number of blocks (**Nblocks**), followed by an empty line.
- For each block, followed by an empty line:
 - **electrode** block: block nature (**electrode**); electrode filled or not (0 or 1); **xmin xmax** on one line; **ymin ymax** on one line; **zmin zmax** on one line; name of the pore-frequency file.
 - **bulk** block: block nature (**bulk**); **xmin xmax**; **ymin ymax**; **zmin zmax**; density of the bulk; frequency of the bulk in ppm.

The density of the bulk must be expressed in the same unit as the densities computed from LAMMPS (number of molecules per $\text{\AA}^3$).

- Three lines, one per fluid species: name of the zmat file and number of molecules, as passed to fftool. The order defines species 1, 2 and 3 (labelled M1, M2 and M3 in the output).
- Next line: index (1, 2 or 3) of the species used by LPC3D for the lattice simulation (**calculated_spec**).
- Next line: name of the LAMMPS equilibration (NPT) script.
- Next line: name of the LAMMPS production (NVT) script.

Example of input file for a supercapacitor (electrode/bulk/electrode):

```
gpu
50 50 150
true true true
1.0
5e-6
1
298
282
1000
0.0
1
500
porouscarbon
psd.txt
3
```

```
electrode
1
1 50
1 50
1 50
pore_freq_1Vneg.txt
```

```
bulk
1 50
1 50
51 100
0.05
0.0
```

```
electrode
1
1 50
1 50
101 150
pore_freq_1Vpos.txt
```

```
Anion.zmat 10000
Cation.zmat 10000
Electrolyte.zmat 30000
1
npt.in.lmp
nvt.in.lmp
```

5.6 Output files

- `simulation_<pore_size>/`: one folder per pore size, containing the fftool, Packmol and LAMMPS outputs of the equilibration run.
- `simulation_<pore_size>/NVT/`: production run, trajectory `positions.xyz` and centres of mass `CM_positions.xyz`.
- `all_densities.dat`: densities of the three species for each pore size, in the two control regions (s1 and s2, corresponding to the negative and positive electrodes). Columns: pore size, M1_s1, M1_s2, M2_s1, M2_s2, M3_s1, M3_s2.
- The outputs of the lattice gas simulation, identical to those of the standalone version.